NEON POKER · БЭКЕНД ОНЛАЙН-ПОКЕРА

Сетевой слой тонкого клиента

Инструкция для Unity-разработчика: что именно должен делать клиент, в каком порядке и на какие грабли не наступить. Кода здесь нет — только устройство слоя и требования к поведению. Точный список сообщений и полей — в PROTOCOL.md, это справочник, который лежит рядом при написании кода.

---

1. Главный принцип: клиент ничего не решает

Сервер — единственный источник правды. Клиент не знает правил покера, не считает банк, не определяет победителя, не проверяет, чей сейчас ход, и не хранит чужих карт — их ему просто не присылают.

Из этого следуют три правила, которые определяют весь дизайн слоя:

1. **Клиент не меняет состояние по своему действию.** Нажали «колл» — отправили сообщение и ждём. Фишки на столе двигаются только когда пришёл `player_action` от сервера. Иначе при отклонённом ходе картинка разъедется с реальностью.
2. **Клиент не додумывает то, чего не прислали.** Нет `options` — кнопок ставок быть не должно, даже если «по логике» ход ваш.
3. **Любое расхождение лечится снимком.** Если что-то пошло не так, клиент просит `request_snapshot` и перерисовывает стол с нуля. Снимок — это всегда полное и достаточное состояние.

Это не перестраховка: сервер отклонит некорректный ход, но UI, который успел показать пользователю несуществующие фишки, чинить уже нечем.

---

2. Из чего состоит слой

Разделите на четыре независимых части. Смешивать их — самая частая ошибка, после которой сетевой код становится нечитаемым.

2.1. Транспорт

Отвечает за сокет и больше ни за что: подключиться, слать и принимать текстовые фреймы, отдавать наружу события «подключено», «сообщение», «отключено», «ошибка». Не разбирает содержимое сообщений, не знает про покер.

Требования:

- WebSocket, текстовые фреймы, UTF-8, JSON. Бинарных фреймов сервер не понимает и ответит `bad_message`.

- Приём в фоновом потоке — Unity не любит блокирующее чтение в главном.
- Есть ровно одно активное соединение. Второй коннект с тем же токеном вытеснит первый, сервер закроет старый с кодом `4001`.

2.2. Очередь в главный поток

Сообщения приходят в фоновом потоке, а трогать `GameObject` можно только в главном. Между транспортом и логикой нужна потокобезопасная очередь: приёмный поток складывает в неё разобранные сообщения, `Update` разбирает.

Это не деталь реализации, а обязательный элемент: без него будут редкие необъяснимые падения Unity, которые крайне трудно ловить.

2.3. Диспетчер сообщений

Разбирает конверт `{"t": …, "d": …, "rid": …}`, находит нужный тип и вызывает обработчик. Здесь же:

- **Неизвестный тип `t` — не ошибка.** Логируем и игнорируем. Сервер будет дополняться, старый клиент не должен падать на новом сообщении.
- **Неизвестное поле в `d` — тоже не ошибка.** Парсер должен пропускать лишние поля, а не валиться.

2.4. Модель стола

Единственное место, где живёт состояние: список мест, стеки, борд, банк, чей ход, свои карты, свои доступные действия. UI читает модель и только её. Обработчики сообщений меняют модель, а не лезут напрямую в интерфейс.

Такое разделение окупается сразу же при реконнекте: пришёл снимок — заменили модель целиком, UI перерисовался. Если бы обработчики правили UI напрямую, пришлось бы вручную сбрасывать каждый элемент.

---

3. Порядок подключения

1. **Получить токен.** `POST /auth/guest` с ником. В ответ — `token`, `player_id`, `balance`.
2. **Сохранить токен на устройстве.** Он живёт 30 суток и служит ключом от аккаунта: потеряли — потеряли и баланс. При следующем запуске отправьте в `/auth/guest` свой прошлый токен, чтобы продлить ту же личность. Передавать в запросе `player_id` нельзя — сервер его не примет: этот идентификатор публичен (соперники видят его в составе стола), и по нему кто угодно выпустил бы токен на чужой аккаунт.
3. **Открыть сокет** на `/ws?token=<токен>`. Альтернатива, если не хотите светить токен в URL: подключиться без параметра и отправить первым сообщением `auth`. До авторизации сервер не примет ничего другого.
4. **Дождаться `welcome`.** В нём приходят профиль, список уровней ставок, размер стола и `action_timeout`. До `welcome` считать сессию установленной нельзя.

5. **Если игрок уже сидел за столом**, сразу следом придёт `table_snapshot` — реконнект отработал сам, ничего дополнительно запрашивать не нужно.

Ошибка авторизации приходит как `error` с кодом `unauthorized`, после чего сервер закрывает соединение с кодом `4401`. Это сигнал «токен протух, получи новый», а не «повтори попытку».

---

4. Состояния клиента

Заведите явный конечный автомат. Без него код обрастает флагами вида `isConnecting`, `hasJoined`, `waitingForAction`, которые рано или поздно разойдутся между собой.

Состояние	Как попали	Что можно делать
`Disconnected`	старт, обрыв, ошибка	подключаться
`Connecting`	открываем сокет	ничего, ждём
`Authorized`	пришёл `welcome`	`quick_join`, `list_stakes`, `ping`
`Joining`	отправили `quick_join`	ждать `joined_table` или `error`
`AtTable`	пришёл `joined_table`	игровые команды
`Reconnecting`	обрыв, есть сохранённый токен	подключаться с задержкой

Переход в `AtTable` делается **только** по `joined_table`, а не по факту отправки `quick_join`. Выход — по `left_table`.

---

5. Ход игрока

Самая ответственная часть. Порядок такой:

1. Пришло `action_required` **с полем** **`options`** — значит, ход ваш. Если `options` нет, это уведомление о чужом ходе: показываем таймер над нужным местом и всё.
2. Кнопки строим **строго по** **`options`**. Не вычисляйте сами, что доступно: правила минимального рейза, неполных олл-инов и «повышать некому, соперник в олл-ине» на клиенте не воспроизвести, а расхождение приведёт к отклонённым ходам и раздражённому игроку.
3. Слайдер ставки ограничивайте `min_amount` и `max_amount` из нужной опции.
4. Отправляем `action`. **В** **`amount`** **идёт суммарная ставка «до» на этой улице, а не приращение.** Если текущая ставка 40, а вы хотите поднять до 120, то `amount = 120`, а не 80. Для `fold`, `check`, `call`, `all_in` поле игнорируется.
5. **Всегда передавайте** **`hand_id`** из `action_required`. Тогда ход, опоздавший к уже закончившейся раздаче, будет отклонён с внятной ошибкой, а не применится к новой.

6. Сразу после отправки — блокируем кнопки и показываем ожидание. Разблокировка по `player_action` со своим `seat_no` или по `error`.

Если пришла `error` с кодом `invalid_action` или `not_your_turn` — состояние клиента разошлось с сервером. Правильная реакция: показать сообщение, запросить `request_snapshot` и перерисоваться.

---

6. Таймер хода

В `action_required` приходят `time_limit` (секунды) и `deadline_ts` (абсолютное серверное время в unix-секундах).

Локальные часы устройства использовать нельзя: они могут отставать на минуты, а на мобильных ещё и прыгают при смене часового пояса. Правильный способ:

- при `welcome` запомнить `server_time` и локальное время в тот же момент — так вы получаете поправку между часами устройства и сервера;
- остаток времени считать как `deadline_ts - (текущее локальное + поправка)`;
- поправку обновлять по `pong` (в нём тоже приходит `server_time`).

Таймер на клиенте — **только индикация**. Решение об истечении времени принимает сервер: он сам сходит за игрока (чек, если можно, иначе фолд) и разошлёт `action_timeout`, а следом обычный `player_action`. Клиент не должен отправлять фолд по своему таймеру: это гонка, в которой можно сбросить руку, которую сервер уже успел зачекать.

Показывайте оставшееся время с запасом: сеть добавляет задержку, и полоска, дошедшая до нуля на клиенте, ещё не означает, что сервер закрыл окно.

---

7. Реконнект

Место держится за игроком 120 секунд (`RECONNECT_WINDOW_SECONDS` на сервере). Раздача всё это время идёт: подошла очередь, а игрока нет — сработает авто-фолд. После трёх авто-фолдов подряд игрока переводят в сит-аут, вернуться можно командой `sit_in`.

Что делает клиент:

1. Обрыв — переходим в `Reconnecting`, показываем оверлей, **не сбрасываем модель стола**: скорее всего, вернёмся на то же место.
2. Переподключаемся с экспоненциальной задержкой (например 1, 2, 4, 8 секунд, потолок 15) и небольшим случайным разбросом. Разброс важен: при обрыве на стороне сервера все шесть клиентов иначе постучатся одновременно.
3. Токен переиспользуем тот же.
4. После `welcome` придёт `table_snapshot` — заменяем модель целиком и убираем оверлей.

5. Если в снимке `you.seat_no` отсутствует, место уже освободили: возвращаемся в лобби и показываем это игроку явно.

Отдельный случай — код закрытия `4001`: это не сетевой сбой, а вход с другого устройства. Переподключаться не нужно, покажите сообщение и остановитесь.

---

8. Что рисовать по каким сообщениям

Событие	Что делает клиент
`hand_started`	сброс борда и ставок, баттон на место, стеки из `players`
`blind_posted`	фишка блайнда на стол, стек уменьшить
`hole_cards`	свои карты; приходит **только вам**
`cards_dealt`	анимация раздачи по порядку мест из `seats`
`action_required`	таймер над местом; кнопки — только если есть `options`
`player_action`	фишки, стек, банк, подпись действия
`action_timeout`	пометка «время вышло», следом придёт `player_action`
`street`	новые карты борда из `cards`, банк из `pot`
`uncalled_bet_returned`	фишки едут обратно игроку
`showdown`	вскрытие: `cards`, `combination`, подсветка `best_five`
`hand_finished`	победители, движение банка, итоговые `stacks`
`seat_update`	состояние одного места: сит-аут, реконнект, докупка
`player_joined` / `player_left`	посадка и уход
`table_state`	«ждём игроков» / «идёт раздача»

Отрисовывайте `board` из `street` целиком, а не накапливайте по одной карте: при потере сообщения накопленный борд разъедется, а полное поле самолечится.

---

9. Подводные камни

Анимация не должна тормозить приём. Сообщения приходят быстрее, чем играются анимации: между `player_action` и `street` может пройти доля секунды. Держите очередь визуальных событий отдельно от очереди сетевых, иначе к концу раздачи накопится отставание. Состояние модели обновляйте сразу, а анимацию проигрывайте вдогонку.

Не привязывайтесь к порядку мест на экране. `seat_no` — это номер места за столом на сервере, от 0 до 5, и он не совпадает с позицией на экране. Своё место всегда рисуется снизу, остальные раскладываются по кругу от него. Держите отдельную функцию пересчёта «серверное место → экранная позиция».

Рейт-лимит — 20 сообщений в секунду. Превышение даёт `rate_limited`. Ловится он легко: `ping` в цикле `Update` вместо таймера, или повторная отправка хода по каждому клику. Ход отправляем один раз и блокируем кнопки.

`join_in_progress` — не повод повторять запрос. Этот код означает, что предыдущий `quick_join` того же игрока ещё выполняется (например, приложение открыли во второй раз). Дождитесь `joined_table` по первому.

Все суммы целые. Дробных фишек в протоколе нет нигде. Не используйте `float` для банка и стеков: округление рано или поздно даст расхождение в одну фишку, и искать его вы будете долго.

Карты — строки из двух символов (`As` , `Td` , `7c`): ранг `2..9 T J Q K A` , масть `c d h s` . Разбирайте в свой enum один раз в слое парсинга, а не в UI.

Пинг нужен. Раз в 10–15 секунд отправляйте `ping` . Он и держит соединение живым через NAT и прокси, и уточняет поправку часов, и даёт понять, что связь пропала, раньше, чем это заметит TCP.

---

10. Порядок работ

Разумная последовательность, каждый шаг проверяем перед следующим:

1. Транспорт + очередь в главный поток. Проверка: подключились, получили `welcome` , `ping` отвечает `pong` .
2. DTO и диспетчер. Проверка: все сообщения разбираются, неизвестные игнорируются без падения.
3. Модель стола и `quick_join` . Проверка: сели за стол, видим соперников (запустите на сервере ботов: `python scripts/bot_client.py --bots 5`).
4. Отрисовка раздачи без ходов: борд, банк, чужие действия, вскрытие.
5. Свой ход: кнопки по `options` , отправка, блокировка, таймер.
6. Реконнект: обрыв, оверлей, восстановление по снимку.
7. Краевые случаи: сит-аут, докупка, выход, ошибки, `4001` .

Проверять всё это можно без второго живого игрока — боты садятся за тот же стол и играют сами.